SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

Teacher guide: Day 8

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

How to help: first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

Day 8: Real shows from the internet fetch and APIs

Yesterday's ShowShelf knew 12 shows. Today it talks to **TVmaze**, a free online database of TV shows. The search box asks TVmaze, the page starts with real shows, and it tells people when it's loading or when something went wrong.

Your code box starts with the finished app from Day 7, tidied into sections. A few lines are new or changed for today; they say **Day 8** in their comments:

- `state.shows` holds the shows on the page. For now it starts with the 12 sample shows from data.js.
- Search-as-you-type is gone: from today the search goes to TVmaze when the form is sent.
- Cards cope with real shows that have no genre or no rating yet.

No internet? The Lab answers with saved sample data from TVmaze, so every step works offline too.

Step 1. What is an API? ~15 min

Teaches: An API answers a web address with data. Template literals `` `...${x}...` `` build text; encodeURIComponent` makes it safe for a URL.

TASK

Under the Day 8 comment at the bottom:

1. Keep the start of every address in a constant: `const API = "https://api.tvmaze.com";`
2. Write a function `searchUrl(term)` that returns the search address for `term`, built with a template literal, `API` and `encodeURIComponent(term)`.
3. Log `searchUrl("the office")` and look at the Console.

COMMON MISTAKES

- Quotes instead of backticks: `"${API}/search..."` keeps the `${...}` as plain text.
- A space or a missing slash: `${API}search/shows`.
- Forgetting `return` in `searchUrl`.

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- Day 8: talking to TVmaze ----------
// Your code for today goes below this line.

// The address of the TVmaze API. Every request starts with it.
const API = "https://api.tvmaze.com";

// The search address: searchUrl("the office") -> ".../search/shows?q=the%20office"
function searchUrl(term) {
  return `${API}/search/shows?q=${encodeURIComponent(term)}`;
}

console.log(searchUrl("the office"));
```

Step 2. Your first fetch ~15 min

Teaches: `await fetch(url)` then `await response.json()`, inside an `async function`.

TASK

Under your `searchUrl` code:

1. Write an `async function tryFetch()` that fetches `searchUrl("office")`, reads the JSON into a constant `results`, and logs `results`.
2. Call it: `tryFetch();`

Look at the Console: a long answer from TVmaze.

COMMON MISTAKES

- Forgetting `await`: you log a promise (`{}`) instead of the data.
- `await` in a function without `async`: "await is only valid in async functions".
- Writing the function but never calling it.

NOTE

If a student's Console says the Lab used sample data, that's fine: every check works offline. Online, the preview asks TVmaze again each time the code runs, so ask students not to hammer "Run again".

A CORRECT ANSWER

```
// ... unchanged ...

console.log(searchUrl("the office"));

// A first request to TVmaze
async function tryFetch() {
  const response = await fetch(searchUrl("office"));
  const results = await response.json();
  console.log(results);
}

tryFetch();
```

Step 3. The shape of the answer ~10 min

Teaches: Read the shape of JSON before using it: a search gives `[{ score, show }]`.

TASK

In `tryFetch`, replace `console.log(results);` with two logs:

1. how many results there are: `results.length`
2. the name of the first show found

COMMON MISTAKES

- `results.show.name`: `results` is an array, so it needs `[0]` first.
- `results[0].name`: the name is inside `show`.

A CORRECT ANSWER

```
// ... unchanged ...
  const response = await fetch(searchUrl("office"));
  const results = await response.json();
  console.log(results.length);
  console.log(results[0].show.name);
}

// ... unchanged ...
```

Step 4. From their shape to ours ~20 min

Teaches: An adapter (`toShow`) turns someone else's data shape into yours; `map` applies it to every item.

TASK

Turn `tryFetch` into a real search function:

1. Rename it to `searchShows` and give it a parameter: `async function searchShows(term)`.
2. Fetch `searchUrl(term)` instead of `searchUrl("office")`.
3. Replace the two logs with `return results.map((result) => toShow(result.show));`
4. Tidy up: delete the test lines `tryFetch();` and `console.log(searchUrl("the office"));`.

COMMON MISTAKES

- `results.map(toShow)`: here each item is `{ score, show }`, so it needs `toShow(result.show)`.
- Keeping `"office"` inside the function instead of `term`.
- Forgetting `return`: the function gives back `undefined`.

A CORRECT ANSWER

```
// ... unchanged ...
}

// Search TVmaze. Gives back a list of shows in OUR shape, ready for createCard.
async function searchShows(term) {
  const response = await fetch(searchUrl(term));
  const results = await response.json();
  return results.map((result) => toShow(result.show));
}
```

Step 5. Search TVmaze ~15 min

Teaches: An `async` function can `await` data, put it in the state and call `render()`.

TASK

1. Fill in the two TODOs in `loadShows`.
2. In the `"submit"` listener (in the Events section), under `event.preventDefault();`, call `loadShows(searchInput.value);`.

Type **friends** in the search box and press Enter.

COMMON MISTAKES

- `state.shows = searchShows(term);` without `await`: the state holds a promise.
- Calling `loadShows` outside the submit listener, so it runs once at start with an empty box.
- Using `searchInput` before it has a value, for example in a `const` at the top.

A CORRECT ANSWER

```
// ... unchanged ...
searchForm.addEventListener("submit", (event) => {
  event.preventDefault();   // don't reload the page
  loadShows(searchInput.value);
});

// ... unchanged ...
  return results.map((result) => toShow(result.show));
}

// Search TVmaze, then show the results on the page
async function loadShows(term) {
  state.shows = await searchShows(term);
  state.genre = "";   // the new shows may not have the chosen genre
  render();
}
```

Step 6. Loading... ~15 min

Teaches: Show a loading message before `await`, and hide it after.

TASK

1. At the top, in the Elements section, find the status line: `const statusMessage = document.querySelector("#status");`
2. At the start of `loadShows`, before the `await`: set its text to "Loading shows..." and show it (`hidden = false`). Also hide `emptyMessage`: an old "nothing found" doesn't belong to the new search.
3. After the `await`: hide the status line again.

Search for something and watch closely: the message flashes up while TVmaze answers.

COMMON MISTAKES

- Hiding the message before the `await`, so it never shows.
- `statusMessage.hidden = "false"` in quotes: any non-empty text counts as true.
- Forgetting the `const statusMessage` line at the top.

A CORRECT ANSWER

```
// ... unchanged ...
const genresBox = document.querySelector("#genres");
const listCount = document.querySelector("#list-count");
const statusMessage = document.querySelector("#status");   // Day 8: "Loading..." and errors

// ---------- State: everything on the page that can change ----------
// ... unchanged ...
// Search TVmaze, then show the results on the page
async function loadShows(term) {
  statusMessage.textContent = "Loading shows…";
  statusMessage.hidden = false;
  emptyMessage.hidden = true;   // an old "nothing found" doesn't belong to the new search
  state.shows = await searchShows(term);
  statusMessage.hidden = true;
  state.genre = "";   // the new shows may not have the chosen genre
  render();
// ... unchanged ...
```

Step 7. When things go wrong ~20 min

Teaches: Check `response.ok`, `throw new Error(...)`, and handle failures with `try / catch`.

TASK

1. In `searchShows`, straight after the `fetch` line: if `response.ok` is false, `throw new Error("TVmaze answered " + response.status);`
2. In `loadShows`, put the `await` line and the lines after it inside `try { }`. In `catch (error) { }`, set the status text to a friendly message and add the class `error`: `statusMessage.classList.add("error");`
3. At the start of `loadShows`, remove the class again, so the next "Loading shows…" isn't red: `statusMessage.classList.remove("error");`

COMMON MISTAKES

- Checking `response.ok` after `response.json()`: an error page may not be JSON, so it crashes first.
- A `catch` that does nothing: the page stays on "Loading…" forever.
- Forgetting to remove the `error` class, so the next loading message is red.

NOTE

Students can see a real error by switching off Wi-Fi and searching (the Lab then uses sample data, so they may need to search a word that isn't in it, like "zzz", to see the empty state instead). The check simulates errors itself.

A CORRECT ANSWER

```
// ... unchanged ...
async function searchShows(term) {
  const response = await fetch(searchUrl(term));
  if (!response.ok) {
    throw new Error("TVmaze answered " + response.status);   // stop here: we have no shows
  }
  const results = await response.json();
  return results.map((result) => toShow(result.show));
// ... unchanged ...
async function loadShows(term) {
  statusMessage.textContent = "Loading shows…";
  statusMessage.classList.remove("error");
  statusMessage.hidden = false;
  emptyMessage.hidden = true;   // an old "nothing found" doesn't belong to the new search
  try {
    state.shows = await searchShows(term);
    statusMessage.hidden = true;
    state.genre = "";   // the new shows may not have the chosen genre
    render();
  } catch (error) {
    // No internet, or TVmaze answered with an error: tell the person, in red
    statusMessage.textContent = "Sorry, the shows could not be loaded (" + error.message + "). Check
your internet connection and try again.";
    statusMessage.classList.add("error");
  }
}
```

Step 8. When nothing is found ~10 min

Teaches: Four states for loaded data: loading, error, empty, success.

TASK

In `loadShows`, inside `try`, before `render();`: set the text of `emptyMessage` to something like `No shows found for "zzz". Try another word.`, with the searched word (`term`) in it. Use a template literal.

Search for **zzz**, then for **dark**.

COMMON MISTAKES

- Quotes instead of backticks around a text with `${term}`.
- Setting the text after `render()` is fine too, but setting it outside `try` shows the old word after an error.

A CORRECT ANSWER

```
// ... unchanged ...
    statusMessage.hidden = true;
    state.genre = "";   // the new shows may not have the chosen genre
    emptyMessage.textContent = `No shows found for "${term}". Try another word.`;
    render();
  } catch (error) {
// ... unchanged ...
```

Step 9. Real shows from the start ~20 min

Teaches: `/shows?page=0` lists shows; `map(toShow)` converts them; `trim()` removes spaces.

TASK

1. **TODO 1** in `getShowsPage`: give back the shows in our shape: `return apiShows.map(toShow);`
2. In `loadShows`, if `term` is `""`, use `await getShowsPage()`; otherwise `await searchShows(term)` as before.
3. In the submit listener, send the trimmed text: `loadShows(searchInput.value.trim());`
4. In `state`, start with no shows: `shows: []`. The page doesn't need data.js any more.
5. Delete the old Start section (the comment and `render();`), and fill in **TODO 2** at the very bottom: `loadShows("");`

COMMON MISTAKES

- `apiShows.map((s) => toShow(s.show))`: page items have no `show` inside.
- Calling `loadShows("")` above the other code, so a `const` isn't ready yet ("before initialization").
- Keeping `shows: shows`, so the sample shows flash before the real ones.

NOTE

Online, page 0 has 250 shows, many without ratings or genres. That's why the Day 8 version of `createCard` checks for them.

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- State: everything on the page that can change ----------
const state = {
  shows: [],      // the shows on the page, from TVmaze (in our shape)
  genre: "",      // the chosen genre ("" means all genres)
  myList: JSON.parse(localStorage.getItem("myList")) ?? [],   // saved ids, or an empty list
// ... unchanged ...
searchForm.addEventListener("submit", (event) => {
  event.preventDefault();   // don't reload the page
  loadShows(searchInput.value.trim());
});

// ... unchanged ...
  emptyMessage.hidden = true;   // an old "nothing found" doesn't belong to the new search
  try {
    if (term === "") {
      state.shows = await getShowsPage();   // empty search box: the first page of shows
    } else {
      state.shows = await searchShows(term);
    }
    statusMessage.hidden = true;
    state.genre = "";   // the new shows may not have the chosen genre
// ... unchanged ...
  }
}

// Page 0 of TVmaze's list of shows: the first 250 (the Lab's sample data has 12)
async function getShowsPage() {
  const response = await fetch(`${API}/shows?page=0`);
  if (!response.ok) {
    throw new Error("TVmaze answered " + response.status);
  }
  const apiShows = await response.json();
  return apiShows.map(toShow);
```

```
}

// ---------- Start: load the first shows from TVmaze ----------
loadShows("");
```

Step 10. Give credit to TVmaze ~10 min

Teaches: Credit your data source; create links with `createElement("a")` and `href`.

TASK

At the bottom of your code:

1. Create an `a` element with the `href` `https://www.tvmaze.com` and the text **Data from TVmaze**.
2. Add it to the paragraph in the footer, after a separator: `document.querySelector(".site-footer p").append(" · ", credit);`

Scroll down in the preview to see it.

COMMON MISTAKES

- Setting `credit.src` instead of `credit.href`.
- Appending to `.site-footer` instead of the paragraph inside it (works, but lands on its own line).

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- Start: load the first shows from TVmaze ----------
loadShows("");

// ---------- Credit: TVmaze's data is free to use if we say where it comes from ----------
const credit = document.createElement("a");
credit.href = "https://www.tvmaze.com";
credit.textContent = "Data from TVmaze";
document.querySelector(".site-footer p").append(" · ", credit);
```

Step 11. My List with TVmaze ids ~15 min

Teaches: When saved data changes meaning, save it under a new key.

TASK

Use the key `"myListIds"` instead of `"myList"` in both places:

1. where `state.myList` is loaded, at the top
2. in `saveMyList`

Add a show, press **Run again**, and check that the counter remembers it.

COMMON MISTAKES

- Changing only one of the two keys: the list saves but never loads (or the other way round).
- A typo in one of the keys, like `"myListID"`.

A CORRECT ANSWER

```
// ... unchanged ...
  shows: [],      // the shows on the page, from TVmaze (in our shape)
  genre: "",      // the chosen genre ("" means all genres)
  myList: JSON.parse(localStorage.getItem("myListIds")) ?? [],   // saved TVmaze ids, or an empty
list
};

// ... unchanged ...
// Save My List in the browser (localStorage keeps text only, so we turn it into JSON text)
function saveMyList() {
  localStorage.setItem("myListIds", JSON.stringify(state.myList));
}

// ... unchanged ...
```

Extra 1. Search as you type, politely ~20 min

Teaches: Debounce: `clearTimeout` then `setTimeout` on every input.

TASK

At the bottom, add an `"input"` listener to `searchInput` that loads the shows for the trimmed text 400 milliseconds after the last letter, with `clearTimeout` and `setTimeout`. Keep the submit listener as it is.

A CORRECT ANSWER

```
// ... unchanged ...
credit.textContent = "Data from TVmaze";
document.querySelector(".site-footer p").append(" · ", credit);

// ---------- Search as you type, but only after a short pause ----------
let searchTimer;
searchInput.addEventListener("input", () => {
  clearTimeout(searchTimer);
  searchTimer = setTimeout(() => {
    loadShows(searchInput.value.trim());
  }, 400);
});
```

Extra 2. Surprise me ~15 min

Teaches: `Math.floor(Math.random() * list.length)` picks a random position.

TASK

1. At the top: `const surpriseButton = document.querySelector("#surprise");`
2. Add an `async` click listener to it that waits for `getShowsPage()`, picks one random show, puts it in `state.shows` **as a list of one** (`[show]`), sets `state.genre` to `""` and calls `render()`.

A CORRECT ANSWER

```
// ... unchanged ...
const listCount = document.querySelector("#list-count");
const statusMessage = document.querySelector("#status");   // Day 8: "Loading..." and errors
const surpriseButton = document.querySelector("#surprise");

// ---------- State: everything on the page that can change ----------
// ... unchanged ...
credit.textContent = "Data from TVmaze";
document.querySelector(".site-footer p").append(" · ", credit);

// ---------- Surprise me: one random show from page 0 ----------
surpriseButton.addEventListener("click", async () => {
  const all = await getShowsPage();
  const index = Math.floor(Math.random() * all.length);
  state.shows = [all[index]];   // still a list, with one show in it
  state.genre = "";
  render();
});
```

Extra 3. How many shows? ~10 min

Teaches: Counts and labels that depend on the state are drawn in `render()`.

TASK

1. At the top: `const resultCount = document.querySelector("#result-count");`
2. In `render()`, after the genre filter, set its text to the number of shows in `list`, followed by " shows".

Click a genre chip and watch the number change.

A CORRECT ANSWER

```
// ... unchanged ...
const listCount = document.querySelector("#list-count");
const statusMessage = document.querySelector("#status");   // Day 8: "Loading..." and errors
const resultCount = document.querySelector("#result-count");

// ---------- State: everything on the page that can change ----------
// ... unchanged ...
  }

  resultCount.textContent = `${list.length} shows`;
  renderGrid(list);
  renderGenres();
// ... unchanged ...
```